CargoTrack v2

Aseguramiento de calidad integrado al pipeline

Yenner Américo Hernández Pérez – 7690-13-3750

Aseguramiento de la Calidad del Software (SQA)
Universidad Mariano Gálvez de Guatemala

4 de noviembre de 2026

Objetivo y alcance

Objetivo

Detectar regresiones antes de incorporar cambios a main, mediante controles repetibles sobre una aplicación existente.

Qué cubre la entrega

Pruebas
Unitarias, de integración y de regresión

Calidad
Análisis estático, cobertura de líneas y ramas, quality gates

Flujo
Revisión por pull request y empaquetado Docker

Documentación
Plan SQA, matriz de pruebas y registro de deuda técnica

Aplicación base: gestión de órdenes, clientes, transportistas, documentos y finanzas.

La entrega incluye CI y artefactos. El despliegue automático queda fuera de esta implementación.

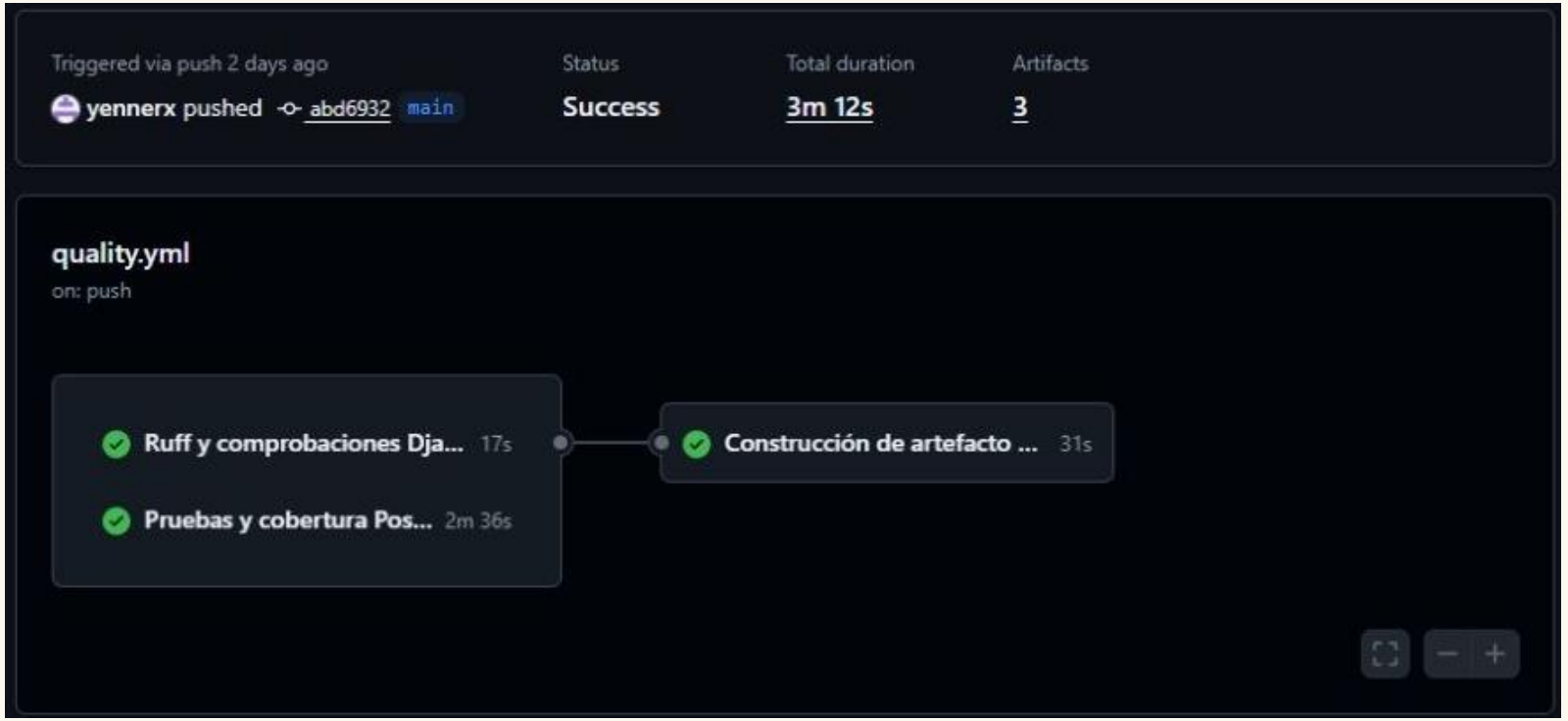
Stack tecnológico

Área	Implementación
Interfaz web	Django Templates, HTMX y Bootstrap 5
Backend	Python 3.12 y Django 5.2
Base de datos	PostgreSQL, versión 16 en CI
Pruebas y cobertura	pytest, pytest-django y pytest-cov
Análisis estático	Ruff y SonarCloud
Automatización y empaquetado	GitHub Actions, Docker y Docker Compose

> La CI usa PostgreSQL efímero y datos de prueba.

Arquitectura del pipeline y quality gates

Cuatro etapas funcionales en tres jobs



> Analyze y Test corren en paralelo. Build depende de ambos; Report publica JUnit, cobertura, Ruff y Docker.

GitHub Actions · run 41580904162 de main

Línea base y criterios de aceptación

Medición inicial: 14 de octubre de 2026, commit 1b47425

Indicador	Línea base	Mínimo configurado
Pruebas aprobadas	138	Suite sin fallos
Cobertura de líneas / sentencias	78,60 %	78,60 %
Cobertura de ramas	57,84 %	57,84 %

Pruebas representativas: comparador, integración con PostgreSQL y regresión de permisos, estados y métodos HTTP.

> **La línea base de ramas inicia debajo del 60 %; el resultado final supera ese umbral.**

Resultados finales y mejora observada

198 pruebas aprobadas
sin fallos, errores ni omisiones

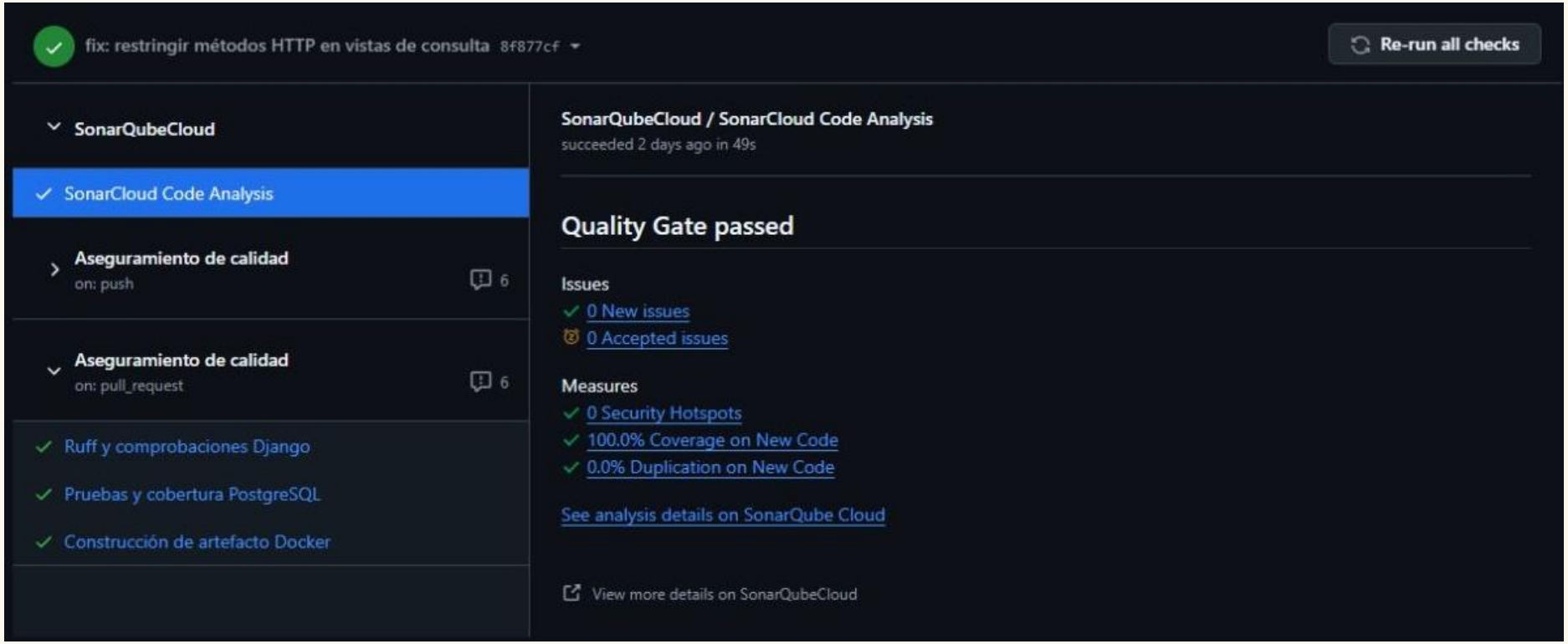
Métrica	Antes	Después	Variación
Pruebas aprobadas	138	198	+60
Cobertura de líneas / sentencias	78,60 %	84,01 %	+5,41 pp
Cobertura de ramas	57,84 %	64,81 %	+6,97 pp

CI final: ejecución 41580904162, commit abd6932, Ruff reportó 0 diagnósticos

pp = puntos porcentuales. La cobertura mide ejecución; no garantiza ausencia de defectos.

PR #6: corrección HTTP y checks aprobados

22 vistas de consulta con require_GET. Las 22 pruebas rechazan POST con HTTP 405.

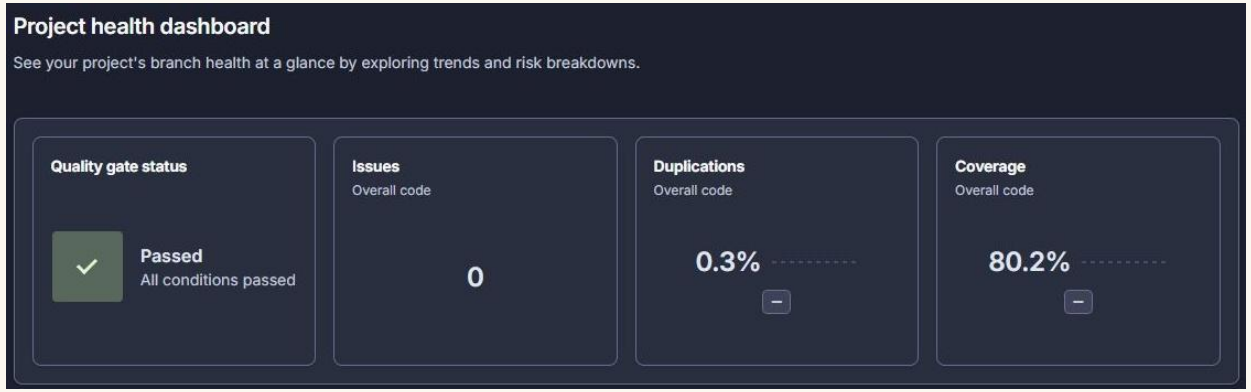


Rama fix/sonar-http-methods, integrado en main como abd6932

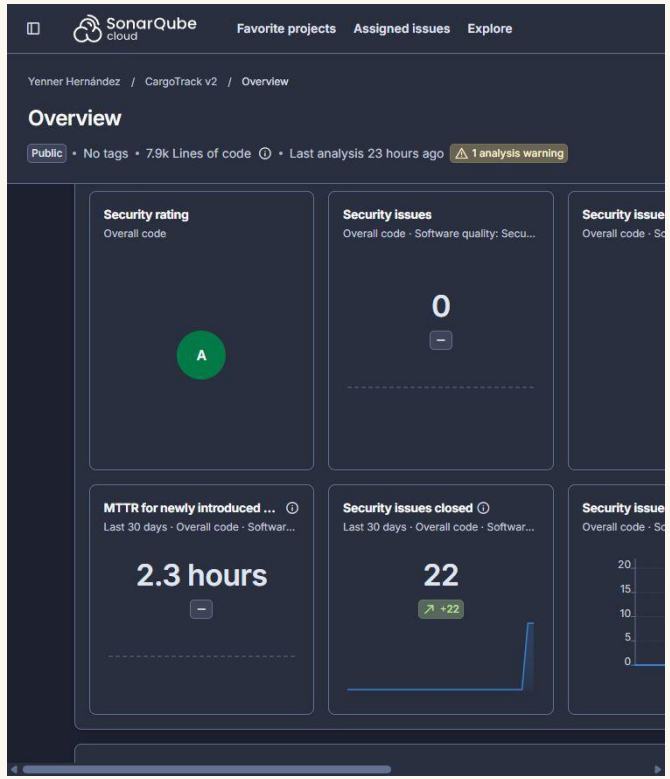
Fuente: github.com/yennerx/CargoTrack_v2-distribucion/pull/6/checks

SonarCloud: métricas y cierre de hallazgos

0 bugs · 0 code smells · 0 vulnerabilidades abiertas



Coverage Sonar: 80,2 %, duplicación: 0,3 %



22 incidencias cerradas

Scope de SonarCloud: apps/. Su cobertura no equivale a líneas y ramas de pytest-cov.

Conclusiones y lecciones aprendidas

01	**CI repetible**	PostgreSQL, migraciones y gates validan cada cambio.
02	**Mejora medible**	Se añadieron 60 pruebas y aumentaron ambas coberturas.
03	**Corrección verificada**	Los 22 hallazgos HTTP quedaron cerrados.

Lección Revisar el contrato de cada endpoint y distinguir la cobertura Sonar de las métricas de pytest-cov.

Deuda pendiente: mediciones de carga (DT-SQA-006), respaldos (DT-SQA-007) y prueba del comando demo (DT-SQA-008).

Referencias y evidencia

Documentación

- Plan SQA
- Matriz de pruebas
- Línea base de calidad
- Reporte de mejora
- Manual técnico
- Guion del video

Ejecuciones y configuración

- PR #6 y revisión automática
- CI del PR, run 41580576891
- CI de main, run 41580904162
- Artefactos del run de main
- Dashboard de SonarCloud
- Workflow de calidad